LAB 1

Project Reset and Architecture Baseline

A verified assessment of the current prototype, production gaps, backlog health, and Senior Project II priorities

Project: flyerrr | Repository: github.com/oanyamele/flyerrr | Prepared: September 10, 2026

Bottom line: Flyerrr is visually strong but functionally local-only. The reset must make the existing screens truthful by adding shared persistence, server-side authorization, and end-to-end verification.

Audit basis

The architecture baseline was established at commit **f0c475a**. Repository and project-board status were refreshed on September 10, 2026 against **main at f6c7136**. Findings were verified with source inspection, build and test commands, GitHub issue metadata, workflow metadata, and project-board status.

1. Executive assessment

All 14 routes render and the mobile-first design system is coherent. Underneath the interface, however, all application data remains in browser storage. There is no shared backend, no trusted authorization boundary, and no server operation that records attendance. The student-to-organizer check-in flow therefore cannot work across two devices, and rewards and reporting remain disconnected from real behavior.

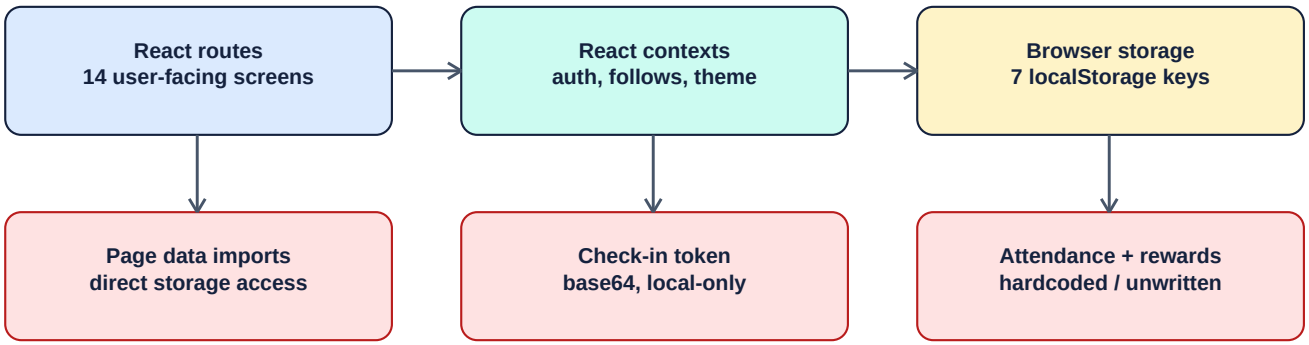
UI and design	Strong	Coherent tokens, responsive layout, light/dark themes
Build	Strong	Production build and TypeScript check completed at baseline
Architecture	Critical gap	No backend or repository seam; pages import storage directly
Core correctness	Critical gap	Attendance is not persisted; rewards and reports use static data
Security	Critical gap	Client-side roles, plaintext credentials, forgeable local QR tokens
Testing	Critical gap	One tautological unit test; defining cross-session flow unverified
Process	Improving	CI and real issues now exist; branch protection is still absent
Backlog	Operational	29 issue-backed cards; expanded 88-row CSV still needs reconciliation
Documentation	Critical gap	Repository README remains a TODO

Current reset progress

The backend stack ADR, relational schema, and Bun standardization are implemented on separate pushed branches and are In Review. They are not yet part of main, so the baseline architecture remains accurate until team approval and merge.

2. Current system architecture

Current architecture



No server, shared database, or trusted authorization boundary

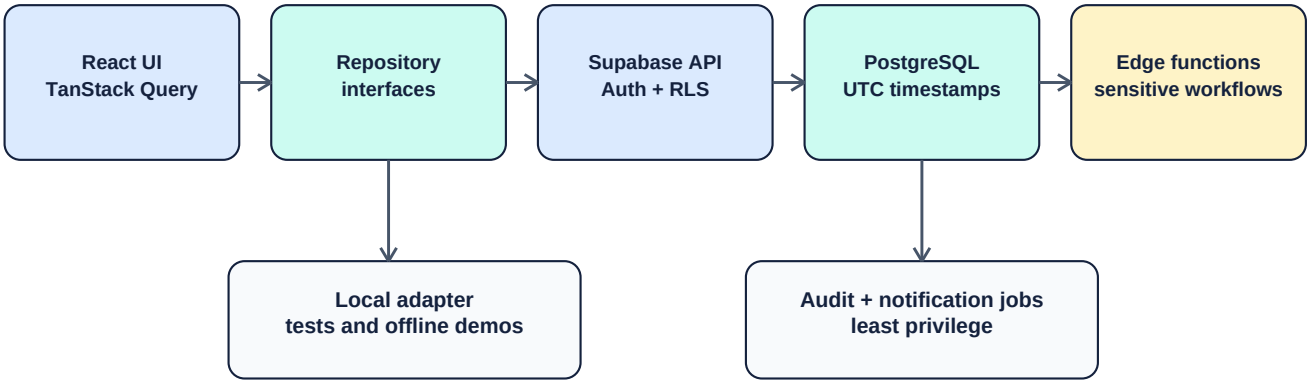
Result: the core student-to-organizer check-in path cannot work across devices.

Presentation	React, React Router, Tailwind, Radix	Fourteen routes and reusable UI components
Client state	Auth, Follow, Theme contexts	Role/session flags, follows, and theme state
Data access	src/data modules	Direct synchronous reads and writes to localStorage
Persistence	Seven localStorage keys	Users, events, RSVP, reminders, tokens, session, theme
Server tier	None	No shared state, trusted auth, transactions, or cross-device flow

Architecture explanation: Routes either read a React context or import storage functions directly. Every write stays on one browser. A QR credential created on a student device is therefore unknown to an organizer device, and successful validation does not append an attendance record.

3. Target architecture

Target architecture



One trusted data model; server-enforced roles; observable failure states

The selected direction uses Supabase as a managed backend: PostgreSQL for shared data, Supabase Auth for credentials, Row Level Security for resource authorization, and narrowly scoped Edge Functions for operations that require server-only secrets or atomic workflows. The React application accesses the backend through repository interfaces and TanStack Query rather than importing persistence modules directly.

Repository interfaces	Allow page-by-page migration and keep a deterministic local test adapter
PostgreSQL	Supports relationships, uniqueness, reporting, transactions, and UTC-capable timestamps
Supabase Auth and RLS	Moves trust off the client and enforces role plus ownership boundaries
TanStack Query	Makes loading, error, caching, and invalidation states explicit
Edge Functions	Contain signed token issuance, privileged transitions, and provider secrets

4. Risk and technical debt register

Severity combines product impact and likelihood. P0 blocks the product from being real; P1 blocks safe development; P2 has compounding cost if deferred.

R-01	P0	Cross-device check-in is impossible	Server-issued, signed, single-use credentials
R-02	P0	Attendance is never recorded	Atomic attendance write and live downstream reads
R-03	P0	Passwords are stored in plaintext	Managed auth; never persist credentials client-side
R-04	P0	No backend exists	Deploy shared API and PostgreSQL staging tier
R-05	P1	Authorization trusts local role flags	Server-side role and ownership enforcement
R-06	P1	QR anti-fraud checks are cosmetic	Sign, expire, bind, and consume token nonces
R-07	P1	React hooks are called conditionally	Move hooks above early return and add regression test
R-08	P1	No meaningful test coverage	Data-layer unit tests and cross-session E2E
R-09	P1	No protected merge gate	Require CI through branch protection
R-10	P1	Several rendered features do not persist	Connect saves, search, verification, and resume data
R-11	P1	Three competing lockfiles	Standardize on Bun; branch is In Review
R-12	P1	Node requirement and setup are undocumented	Add engines, .nvmrc, and complete README
R-13	P1	Dependency vulnerabilities were reported	Upgrade, audit, and enforce the gate
R-14	P2	Event dates are display strings	Use timestamptz and explicit display timezones
R-15	P2	Substantial dead code	Remove unreferenced components after usage verification
R-16	P2	Large initial JavaScript bundle	Route-level code splitting and size budget
R-17	P2	TypeScript strictness is disabled	Enable strict checks and fix fallout
R-18	P2	Generated-platform coupling	Remove tagger and document owned architecture
R-19	P2	Installed libraries are unused	Adopt deliberately during backend migration or remove

P0 - Blocking	4	34 points
P1 - Safe-development blockers	9	30 points
P2 - Compounding cleanup	6	17 points
Total	19	81 points

5. Backlog health and readiness

Live GitHub issues	29 total
Project status	25 Backlog, 3 In Review, 1 Done
Technical-debt coverage	13 issues labeled tech-debt
Milestones	4
Labels	23, including priority, feature, security, testing, and debt
Backlog source	88 detailed feature rows in docs/backlog.csv
CI	One workflow exists on main
Branch protection	Not enabled; GitHub API returns 404
Historical PR traceability	Four merged PRs; none linked to the new issue workflow

Assessment: 46/50 - operational with a completeness gap. The board now uses real, assigned, labeled, estimated issues with acceptance criteria. The next governance task is to reconcile the 29 live issues with the expanded 88-row backlog, require issue links in new pull requests, and protect main with the CI check.

Evidence of improvement

Done	Convert draft cards to Issues	Issue #5 closed; 29 issue-backed board cards
In Review	Standardize on Bun	Issue #9; pushed implementation branch
In Review	Relational schema with ISO timestamps	Issue #13; ERD and policy document
In Review	Backend stack ADR	Issue #33; Supabase decision record

6. Initial Senior Project II priorities

Guiding principle: Stop adding screens. Make the screens that exist tell the truth.

Sprint 0 - Foundation	CI, branch rules, Bun, Node docs, hook fix, dependency repair	Green protected check and reproducible setup
Sprint 1 - Backend and seam	ADR, relational schema, staging backend, repositories, auth, TanStack Query	Three separate devices observe the same published event
Sprint 2 - Core loop	Signed check-in tokens, attendance writes, rewards/reports, saves, verification persistence	One cross-session check-in creates one attendance record and updates points
Sprint 3 - Hardening	Coverage, E2E, strict TypeScript, accessibility, failure states, bundle budget	CI proves correctness, resilience, and usability before merge

Explicit deferrals

New screens, visual redesign, real NFC, and push notifications are not current priorities. QR must work across devices first; notifications depend on shared server state; and the existing interface is already the strongest part of the product.

Known unknowns and team risks

Team ownership of generated data code is uneven	Human reviewer for data and context changes; code-reading session
Backend implementation is new scope	Timebox decisions; prefer documented managed services
ADR and schema are not yet approved	Review and merge contracts before implementation begins
Expanded backlog may exceed capacity	Validate dependencies, preserve P0 ordering, defer lower-value P2 work

7. Conclusion and evidence trail

Flyerrr enters Senior Project II with a real product asset and a real architectural liability. The asset is a coherent mobile experience. The liability is that the application has no shared source of truth and no trusted execution boundary. The backlog reset is now operational; the architecture reset is defined and waiting for reviewed implementation.

> **Semester success criterion: a student checks in from one session, an authorized organizer verifies from another, exactly one attendance record is persisted, and the student's points and the organizer's report update from that same server record.**

Repository evidence

docs/PROJECT_RESET_REPORT.md	Full audit, findings, board assessment, and priorities
docs/architecture.md	Version-controlled Mermaid source for current and target diagrams
DEBT_AND_RISK.md	Rubric-aligned debt and AI-risk inventory
docs/backlog.csv	Feature-level owners, dependencies, estimates, and acceptance criteria
.github/workflows/ci.yml	Automated typecheck, lint, test, build, and audit workflow
GitHub Issues #5-#33	Live issue-backed project board

Reproduction commands include Bun install, TypeScript checking, production build, Vitest, ESLint, dependency audit, GitHub issue listing, workflow inspection, and branch-protection inspection. The full command trail is retained in the Markdown report for repeatability.